Technical Report

Makan Map Crowd Level Prediction System: An End-To-End Pipeline

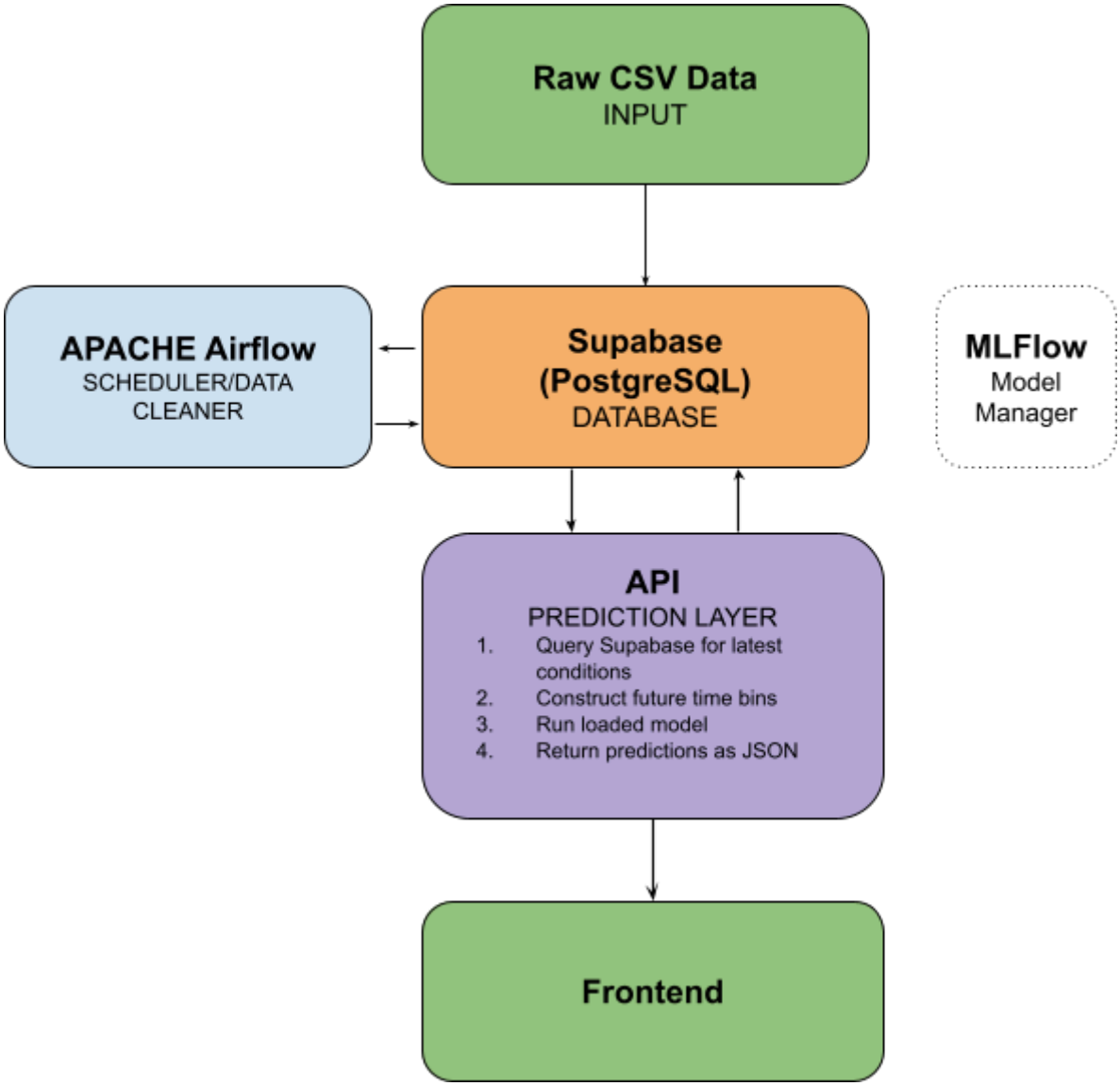
1. Introduction

This technical report presents the design and implementation of the **Makan Map Crowd Level Prediction System**, a machine learning-driven platform developed to predict crowdedness levels at food court locations across Singapore. The system aims to assist customers in planning their visits by avoiding peak periods while also enabling food court operators to optimize resource allocation and operational efficiency.

Food courts in Singapore experience significant fluctuations in crowd density due to factors such as time of day, day of the week, weather conditions, holidays, and consumer behavior. Without predictive insights, customers may experience long queues and waiting times, while vendors struggle to manage staffing and inventory effectively. To address these challenges, the proposed system leverages historical transaction data and contextual information to forecast crowd levels and waiting times.

The platform integrates data engineering pipelines, machine learning models, and modern MLOps practices to deliver a scalable and production-ready solution. Data is ingested from a single source, processed through automated ETL pipelines orchestrated using Apache Airflow and stored in Supabase as the centralized data repository. Machine learning models are trained on historical and contextual data, including temporal patterns and external factors, to predict crowdedness levels and are deployed through a real-time API service for efficient access by applications and stakeholders.

2. System Architecture Overview



3. Data Pipeline & EDA

The data pipeline is designed to transform raw and potentially noisy data into structured, machine-learning-ready features used for predicting food court crowdedness. There is a proper separation between ingestion, processing and model-preparation. Data processing and orchestration are handled using Apache Airflow, which manages the execution of data cleaning and feature engineering tasks.

3.1 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to understand the characteristics and quality of the collected dataset before building the automated data pipeline. The goal was to identify inconsistencies, data quality issues, and patterns that could influence the machine learning model.

Initial analysis of the raw dataset revealed several common issues such as duplicate records, inconsistent timestamp formats, missing values, and numeric fields containing special characters. These findings informed the design of the cleaning rules later implemented in the automated pipeline.

The dataset includes attributes such as location identifiers, timestamps, weather information, temperature, humidity, public holiday indicators, and temporal features such as day of week and hour of day. These variables were analyzed to determine their relevance for predicting crowdedness levels. The findings guided the design of the automated data cleaning pipeline.

3.2 Database Schema Design

The system implements a three-layer database structure to ensure data reliability, traceability, and compatibility with machine learning workflows.

Stage 1: Raw Data Layer
The raw_data table stores all ingested records in their original format. Fields are intentionally stored as TEXT or FLOAT without strict constraints to prevent data rejection during ingestion and to preserve dirty or inconsistent data for auditing and debugging.

The table includes attributes such as location identifiers, timestamps, weather information, and temporal features like day of week and hour of day. This layer acts as the source of truth, maintaining the original dataset before any transformations are applied.

Step 2: Clean Data Layer
The clean_data table contains the processed version of the raw dataset after data cleaning and validation. At this stage, proper data types are enforced, with numeric fields such as latitude, longitude, temperature, and humidity converted to floating-point values, and categorical indicators such as weekends and public holidays converted to Boolean values.

The record_id is defined as the primary key to ensure record uniqueness.

Business rules are also applied, including filtering records to only include food court operating hours between 6 AM and 10 PM. Temperature values are standardized to Celsius, and missing values are handled using location-based statistical imputation, resulting in a consistent and reliable dataset for further processing.

Step 3: Feature Data Layer
The final stage stores machine-learning-ready features in the features table. This dataset includes engineered variables derived from the cleaned data, such as cyclical encodings for time-based features and aggregated location-level metrics.

Time features such as hour of day and day of week are transformed appropriately with additional features that include weather category encoding, location popularity metrics and Boolean indicators for weekends and holidays.

The dataset also includes the **target variable (crowd_level)** used for model training and evaluation.

3.3 Airflow Data Pipeline

The data processing workflow is orchestrated using the **Apache Airflow Directed Acyclic Graph (DAG)** named **crowd_level_predictor_pipeline**. The DAG coordinates the execution of the main data transformation stages and ensures tasks run in the correct order.

The pipeline consists of two primary tasks:

- **Data Cleaning Task:** Processes raw records from the raw_data table and generates structured records in the clean_data table.
- **Feature Engineering Task:** Converts cleaned records into machine-learning-ready features stored in the features table.

The pipeline is designed to be manually triggered, ensuring that data engineers can validate pipeline runs before executing production workflows.

3.4 Data Cleaning Process

This automated data cleaning pipeline implements the transformations identified during the exploratory data analysis phase.

- **Duplicate Removal**: Duplicate records are removed using the record_id to maintain data integrity.
- **Timestamp Standardization**: Timestamps are converted into a consistent format, with missing values handled.

- **Numeric Data Cleaning**: Fields such as temperature and humidity are cleaned using regular expressions to remove special characters and converted into numeric types.
- **Missing value handling**: Missing numeric values are filled using location-based median imputation, preserving location-specific patterns.
- **Boolean Standardization**: Fields such as public holiday indicators are standardized to consistent True/False values.
- **Temperature normalization**: Temperature values detected in Fahrenheit are converted to Celsius to maintain consistency.
- **Business Rule Filtering**: Records outside valid food court operating hours (6 AM – 10 PM) are removed.

3.5 Feature Engineering Process

- **ML Feature Transformation:** Cleaned data is transformed into structured features suitable for machine learning models.
- **Cyclical Time Encoding:** Temporal features such as hour of day and day of week are encoded using sine and cosine functions to preserve the cyclical nature of time.
- **Weather Encoding:** Weather conditions are converted into categorical numerical values for model compatibility.
- **Location Popularity Feature:** A location popularity metric is added based on historical visit frequency to capture inherent location demand.
- **Behavioral Indicators:** Boolean features such as weekend and public holiday flags are included to capture customer activity patterns.
- **Efficient Processing:** The pipeline uses vectorized DataFrame operations to efficiently transform large datasets.

Approximately 200,000 records are processed in seconds, producing an **11-feature dataset** optimized for model training.

4. ML Model

4.1 Model Development Approach

Two modeling approaches were explored to determine the most suitable method for predicting food court crowdedness.

Experiment 1: Regression Approach
The first experiment attempted to predict a continuous crowdedness score normalized between 0 and 1. The crowd score was derived by normalizing the event count per 30-minute time bin against the observed minimum and maximum counts in the dataset.

Setup:

- Time binning: 30-minute intervals
- Target variable: Continuous crowd score (0.0 – 1.0), derived from normalized event counts per location per time bin
- Features used: temperature, humidity, is_weekend, is_public_holiday, location_freq, weather, location_id, time_block, day_name
- Train/test split: Time-based 80/20 split
- Models tested:
    - Gradient Boost Regressor
    - Random Forest Regressor (400 trees)
    - XGBoost Regressor (800 trees)

Results:

Model (Regressors)	MAE	RMSE	R^2
Gradient Boosting	0.0901	0.1189	0.239
Random Forest	0.0870	0.1193	0.233
XGBoost	0.0861	0.1175	0.259

Analysis:

- **Gradient Boost delivered the highest accuracy**:
    - MAE: **0.0901**
    - RMSE: **0.1189**
    - R^2: **0.239**
- **Random Forest** was competitive on MAE (**0.0870**) but scored lower on R^2.
- **XGBoost** was the weakest performer across all metrics except for R^2.
- **R^2 scores of 0.23–0.26** across all models show that current features explain only about a quarter of the variance in crowd scores.
- This limitation stems from missing influential factors such as meal-period effects, promotional events, and stall-level popularity rather than model shortcomings.
- Additionally, comparing the accuracy and F1 scores as well as the mean crowd score of the total dataset (0.1265) shows that the data is imbalanced to lower crowds which has affected the model. This can be easily remedied by retraining on better data or using our scripts to train a new model on balanced data.
- MLflow logged all parameters, metrics, and model artifacts for every run, enabling systematic comparison and full reproducibility.

Experiment 2: Classification Approach
The second experiment framed the problem as a multi-class classification task, predicting one of three crowd levels: Low, Medium, or High.

Setup:

- Time binning: 30-minute intervals
- Target variable: Crowd level category (Low / Medium / High), derived using quantile-based binning of event counts per time bin
- Features used: day_of_week, hour_of_day, is_weekend, is_public_holiday, temperature, humidity, weather
- Crowdedness levels: Low, Medium, High
- Binning method: Quantile-based segmentation
- Train/test split: Time-based 80/20 split

The preprocessing pipeline included:

- Median imputation for numeric features
- Most frequent value imputation for categorical features
- One-hot encoding for categorical variables

Models tested and Results:

Model	Accuracy	F1 Macro
Gradient Boosting	0.932	0.364
Random Forest	0.925	0.388
XGBoost	0.927	0.368

Analysis:

- **Accuracy scores of 0.925 - 0.932** are **misleading** because quantile-based binning created a dataset heavily skewed toward the Low crowd class, as most 30-minute windows had few scan events. This imbalance means a model that mostly predicts Low can achieve high accuracy without learning meaningful patterns for Medium or High crowd periods.
- **F1 Macro scores of 0.364 - 0.388** provide a more truthful assessment, since they weigh all classes equally.
- These F1 scores show that **all three models struggled** with Medium and High predictions due to **insufficient representation** of those classes in training data.
- Performance differences between the models were negligible, confirming that data imbalance, not model choice - was the primary bottleneck.

4.2 Model selection

- The **regression approach** was chosen for production because:
  - The deliverable required a **continuous 0 - 1 crowd score**, not discrete classes.
  - The classification models' high accuracy was distorted by **class imbalance**, and their F1 Macro scores showed unreliable Medium/High predictions.
- Among the three regression models, **Gradient** performed best based on MAE and RMSE, though overall differences were small.
- The main improvement opportunity lies in data quality, not model tuning, specifically:
  - The skew toward low-crowd periods.
  - Missing features capturing stall-level or event-level demand signals.
- The final model is saved as a **serialized sklearn pipeline (joblib)**, bundling preprocessing and the trained Gradient Boost model to ensure consistent OneHotEncoding and feature alignment between training and production.

4.3 Training Data Preparation

The dataset was split using a time-based 80/20 split rather than a random split. A time-based split simulates real-world deployment conditions, where the model is always predicting future time periods it has never seen, and produces a more honest assessment of generalization performance.

- Training set**:** First 80% of the dataset
- Test set: remaining 20% of the dataset

All preprocessing from the preprocessing pipeline was fitted exclusively on the training set and applied to the test set, ensuring no information from the test period influenced feature transformations.

5. Prediction API

The prediction layer is implemented as a REST API using **FastAPI**, chosen for its automatic request validation via Pydantic schemas and its native support for type hints, which reduces the chance of malformed inputs reaching the model.

The API exposes a single `/predict` endpoint that accepts a POST request containing the current conditions at a location: temperature, humidity, weather, location frequency, and whether it is a public holiday. From these inputs, the API constructs the next N 30-minute time bins automatically, deriving `time_block`, `day_name`, and `is_weekend` from the actual future timestamps rather than requiring the frontend to compute them. Weather, temperature, and

humidity are carried forward across all bins, which is a reasonable assumption for short forecast windows of up to 3 hours.

The model artifact is loaded once at startup from a `.pkl` file containing both the trained sklearn pipeline and the `bin_edges` thresholds used during training. Loading at startup rather than per-request avoids the overhead of deserialising the model on every call. The full pipeline, preprocessor and model together, is saved and loaded as a single object, ensuring that OneHotEncoding of categorical variables (`weather`, `location_id`, `time_block`, `day_name`) is applied consistently at inference time using the exact same encoder fitted during training.

The raw model output (class 0, 1, or 2) is normalised to a 0.0 - 1.0 crowdedness score and returned alongside a human-readable label (Low / Medium / High) and the timestamp of each bin, giving the frontend everything it needs to display a short-horizon forecast without any further processing.

A `/health` endpoint is also exposed for Docker container health checks to confirm the model loaded successfully on startup.

6. Limitations & Future Works

6.1 Queue Length & Wait-Time Forecasts

The original project brief called for predictions of queue length and wait times at the stall level. Due to time constraints, the delivered model predicts crowd level (Low/Medium/High) only. As a practical workaround, future works could derive rough queue length and wait-time estimates directly from the crowd level output; for example, mapping Low/Medium/High to estimated queue ranges based on domain knowledge from the industry partner. This would allow the core deliverable to be approximated without requiring a separate model.

6.2 Imbalanced Training Data

The crowd level target was constructed by binning timestamps into 30-minute windows and counting the number of scan events per location per bin. In practice, most locations had relatively few scans within any given 30-minute window, resulting in a dataset heavily skewed toward Low crowd levels. This imbalance is reflected in the model evaluation results, high accuracy but low F1 Macro, indicating the model performs well on the majority class but struggles to correctly predict Medium and High crowd levels. Future works could address this through oversampling techniques such as SMOTE, collecting more data across peak periods, or adjusting class weights during training. The existing pipeline scripts are designed to be reusable, and retraining on more balanced data should be straightforward once richer data is available.

6.3 Automated Model Retraining via MLflow

The current implementation uses MLflow for experiment tracking and model artifact storage during manual training runs. An automated retraining pipeline, where the model periodically retrains on new data ingested through Airflow and registers updated artifacts in MLflow, was not implemented due to time constraints and limited access to a live Supabase instance during development. The groundwork for this exists in the current codebase, and future works can extend the existing Airflow DAG to include a retraining task that triggers when sufficient new data has accumulated.

6.4 Frontend

The frontend implementation was scoped down due to time and computational constraints. Features that were cut include side-by-side model prediction comparisons, a restaurant location map, and richer visualisation components such as crowd trend graphs over time. The prediction API is designed to support these features. The prediction API returns per-bin timestamps, crowd labels, and scores sufficient to support basic crowd trend visualisations without backend changes. However, additional features such as a location map or model comparison views would require new API endpoints to expose geospatial data and multi-model outputs respectively.

7. Conclusion

The Makan Map Crowd Level Prediction System is a full end-to-end machine learning pipeline for a real urban challenge in Singapore. It transforms noisy, inconsistent raw data into reliable crowd-level forecasts served through a production-ready REST API. Each stage: ingestion, cleaning, feature engineering, model training, and deployment - is built for reproducibility and easy extension.

The automated Airflow pipeline successfully processes approximately 200,000 records through two sequential stages, transforming raw event-level data into an 11-feature, machine learning-ready dataset with minimal manual intervention. Among the three regression models evaluated, XGBoost achieved the best overall performance with an MAE of 0.091. The F1 Macro scores further highlight the practical challenge of predicting Medium and High crowd levels given the skewed distribution of training data toward Low crowd periods.

Despite its modeling constraints, the API provides a usable short-horizon service, outputting crowd scores and labels in rolling 30-minute intervals without requiring the frontend to compute time-based features. MLflow tracks all model runs for reproducibility and sets the stage for future automated retraining.
Key improvements include fixing training data imbalance via oversampling or richer peak-period collection, extending the Airflow DAG for periodic retraining, and adding the frontend's

geospatial mapping and crowd-trend visualizations already supported by the API. These upgrades would move the system from proof-of-concept to a fully operational tool for patrons and operators.